

AI FitTrack – Project Document

Personalized fitness recommendations powered by AI

Project Overview

The FitTrack AI backend is a secure, scalable RESTful application built with Node.js, Express.js, MongoDB, and Mongoose using the Model-View-Controller (MVC) architecture. It exposes standardized JSON APIs for workout management, search, and user authentication secured via bcrypt.js password encryption and JSON Web Token (JWT) middleware. Integrated with Google Gemini AI, the system delivers personalized workout recommendations based on user profiles and goals while analyzing metrics like workout duration and calories burned to generate actionable fitness insights. Complete with robust input validation, error handling, and a modular service design, the backend is built for reliable integration across web, mobile, and third-party platforms.

Team Members

Team ID: SWTID-2026-2409

Class: III B.Sc Computer Science, Shift - I

Name	Email	NM ID
Kavin R	kavinrajendran1437@gmail.com	34E40910C481BEEA7434CD00FB15C37D
Mohanaprasath S	saravanansmp2007@gmail.com	8B1054375BA6690BF274482938D9EFAE
Mouneesh K	mouneeshkannan14@gmail.com	BAF27C4356E93A4923D46FC818268E56
Nishanth K	nishanthk072007@gmail.com	8D9EEA502C942A7A22A4B0DD14BCECA0

Phase 1

Brainstorming & Ideation Phase

Phase 1: Brainstorming & Ideation Phase

Background

Rahul is a fitness enthusiast who exercises regularly but records his workouts manually using notebooks and spreadsheets. Although he maintains consistency in his fitness routine, he finds it difficult to organize workout history, monitor progress, and determine whether his exercise habits align with his fitness goals. He also lacks professional guidance when deciding which workouts are best suited for his current fitness level.

Problem

- Difficulty maintaining a centralized record of workouts.
- Manual tracking leads to inaccurate workout history.
- No secure user authentication for personal fitness records.
- Unable to search previous workouts efficiently.
- No personalized workout recommendations.
- No insights regarding overall workout performance.
- Time-consuming manual calculation of calories burned and workout duration.

Solution

The **FitTrack AI** provides a centralized backend system where users can securely register, authenticate, and manage workout records. Users can add, update, delete, retrieve, and search workouts through RESTful APIs. By integrating **Google Gemini AI**, the application analyzes user information and workout statistics to generate intelligent workout recommendations and personalized fitness insights.

The backend follows industry-standard security practices including JWT authentication, password hashing, middleware-based authorization, centralized error handling, and MongoDB schema validation.

User Usage

Users can register and log in with their email and password to receive a JWT token, granting access to protected APIs to:

- Create, view, update, and delete workout records, or retrieve specific workouts by ID.
- Search and filter workouts by name, category, or date.
- Generate personalized, AI-powered workout recommendations.
- Obtain AI-generated fitness insights based on workout statistics.

AI Usage

The AI module connects to Google Gemini through a dedicated service layer to deliver two core capabilities:

- **Workout Recommendations:** Processes user inputs (age, fitness goal, and experience level) to generate personalized workout plans, weekly exercise suggestions, difficulty-tailored recommendations, motivational guidance, and safety tips.
- **Fitness Insights:** Analyzes workout metrics (total workouts, average duration, and total calories burned) to produce personalized performance insights.

Outcome

The FitTrack AI backend uses a layered Model-View-Controller (MVC) architecture that separates routing, middleware, business logic, database operations, and AI services into independent modules with single responsibilities. Incoming HTTP requests to the Express server are directed to specific routes, passing through JWT authentication middleware on protected endpoints before reaching the controllers. The controllers then coordinate with Mongoose models for data persistence and a dedicated Google Gemini AI service layer to process requests and return structured JSON responses, ensuring high security, maintainability, and scalability.

Phase 2

Requirement Analysis Phase

Phase 2: Requirement Analysis Phase

Software Requirements

The FitTrack AI requires the following software components for development and execution.**Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).

- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

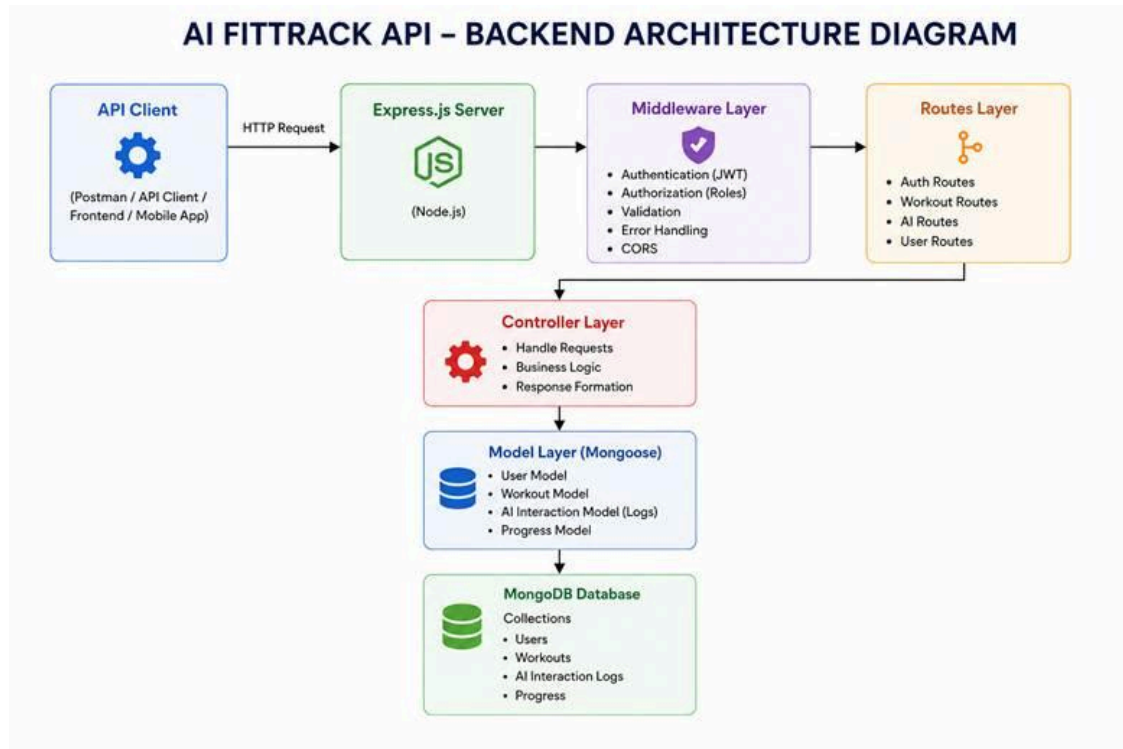
Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Phase 3

Project Design Phase Phase

Phase 3: Project Design Phase



The FitTrack AI backend uses a layered Model-View-Controller (MVC) architecture that separates routing, middleware, business logic, database operations, and AI services into independent modules with single responsibilities. Incoming HTTP requests to the Express server are directed to specific routes, passing through JWT authentication middleware on protected endpoints before reaching the controllers. The controllers then coordinate with Mongoose models for data persistence and a dedicated Google Gemini AI service layer to process requests and return structured JSON responses, ensuring high security, maintainability, and scalability.

Overall Architecture Components

- **Client & Routing Layer:** Client Application (React, Mobile App, Postman), Express Server, API Routes, and Authentication Middleware.
- **Logic & AI Layer:** Controllers, Services, and Google Gemini AI Service.
- **Data Layer:** Mongoose Models and MongoDB Database.

All components communicate through well-defined interfaces, ensuring loose coupling and easier future enhancements.

Phase 4

Project Planning Phase

Phase 4: Project Planning Phase

Components Definition

The FitTrack AI follows a modular backend architecture in which every component performs a dedicated responsibility. The separation of concerns ensures maintainability, scalability, and efficient execution of requests. Each request passes through multiple layers before reaching the database or AI service, ensuring secure and reliable processing.

1. Client Layer (Frontend / API Consumer)

Acting as the application entry point (React web app, mobile app, or Postman), the Client Layer sends HTTP requests to the backend and renders the returned JSON responses.

- **Responsibilities:** Sends API requests, stores the JWT token post-login, invokes AI endpoints, and displays workout data alongside success or error messages.
- **APIs Accessed:** Authentication, Workout Management, AI Recommendations, and AI Fitness Insights.

2. Express Server Gateway

Acting as the central gateway, the Express server listens for incoming HTTP requests, configures middleware, and directs traffic to the appropriate controllers.

- **Responsibilities:** Starts the HTTP server, parses JSON request bodies, enables CORS, connects middleware, registers routes, and handles centralized errors.
- **Request Workflow:** Receives HTTP requests, validates request formats, forwards requests to routes, and returns structured JSON responses.

3. Authentication Middleware

The Authentication Middleware protects secured API endpoints by validating the JSON Web Token (JWT) included in the request's Authorization header.

- **Responsibilities:** Reads the Authorization header, extracts and verifies the JWT signature, decodes and attaches user data to the request object, and rejects unauthorized requests.
- **Benefits:** Prevents unauthorized access, secures workout data for authenticated users only, and eliminates repetitive authentication logic inside controllers.

4. Route Layer

The Route Layer defines the application's API endpoints and maps incoming HTTP requests to their corresponding controller methods.

- **Authentication Routes:** Register User, Login User, and User Profile.
- **Workout Routes:** Add Workout, Get All Workouts, Search Workouts, Get Workout by ID, Update Workout, and Delete Workout.
- **AI Routes:** Workout Recommendation and Fitness Insights.

5. Controller Layer

Controllers manage core application logic by receiving routed requests, validating input data, invoking services or database operations, and returning standardized JSON responses.

- **Authentication Controller:** Handles user registration, login, and profile retrieval.
- **Workout Controller:** Manages creating, retrieving, updating, deleting, and searching workout records.
- **AI Controller:** Coordinates sending prompts to Google Gemini, receiving AI-generated recommendations, and returning responses to the client.

6. Service Layer

The Service Layer encapsulates reusable business logic shared across multiple controllers.

- **Gemini AI Service:** Prepares and sends prompts to Google Gemini, processes AI-generated responses, and returns formatted recommendations.
- **JWT Service:** Generates JWT tokens, configures token expiration, and verifies user identity.
- **Password Service:** Hashes passwords prior to storage and compares credentials during login.

7. Database Layer

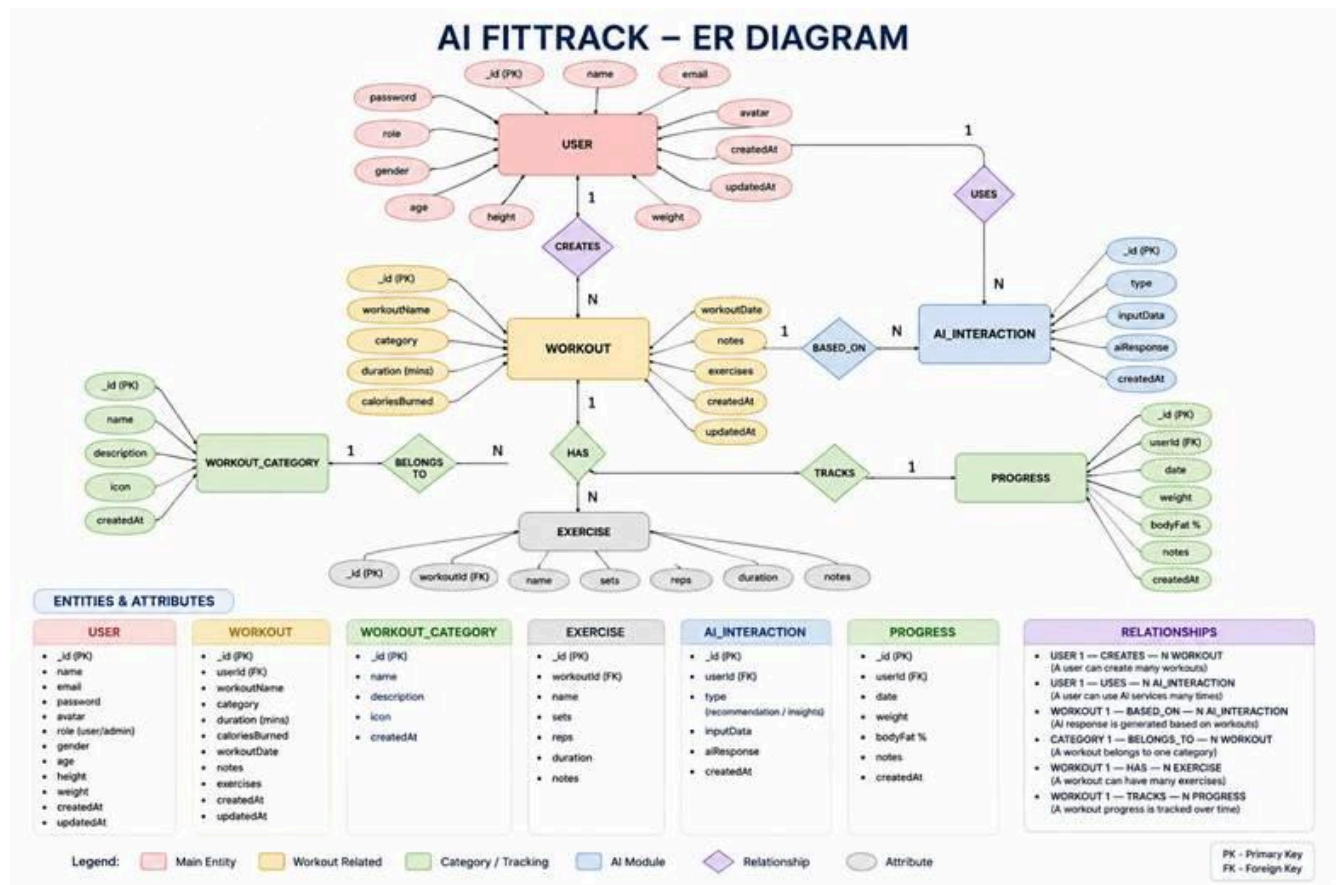
MongoDB provides persistent data storage, while Mongoose serves as the Object Data Modeling (ODM) library to streamline communication between Node.js and the database.

- **Collections:** Users Collection and Workouts Collection.
- **Advantages:** Offers a flexible schema, high performance, easy data retrieval, and a scalable architecture.

Entity Relationship (ER) Diagram

Description

The AI FitTrack backend consists of two primary collections that are logically connected through a one-to-many relationship.



1. User Entity

The User entity stores and manages user authentication credentials and profile information in the database.

- **Attributes:** Includes a primary key (_id as ObjectId), a mandatory user name (name as String), a unique email address (email as String), an encrypted password (password as String), and automated record creation and update timestamps (createdAt and updatedAt as Date).
- **Constraints & Security:** Enforces a mandatory name and unique email address, hashes passwords using bcrypt before storage, and generates a JWT token upon successful login.

2. Workout Entity

The Workout entity stores workout-related information for each authenticated user.

Attributes

Field	Type	Description
_id	ObjectId	Primary Key
user	ObjectId	References User Collection
workoutName	String	Workout Name
category	String	Workout Category
duration	Number	Duration (Minutes)
caloriesBurned	Number	Calories Burned
workoutDate	Date	Workout Date
createdAt	Date	Created Timestamp
updatedAt	Date	Updated Timestamp

Relationship

User → Workout

Relationship Type: One-to-Many

A single user can create multiple workout records. Each workout belongs to only one authenticated user.

Example: John Doe

- Morning Running
- Evening Jogging
- Upper Body Strength
- Cycling Session

This relationship is maintained using the **user** field inside the Workout schema, which stores the ObjectId of the corresponding user.

Key Features

The FitTrack AI provides a complete backend solution for workout tracking and AI-assisted fitness guidance.

1. Secure Authentication:

Implements JWT-based security encompassing user registration, secure login, password encryption, JWT generation, and protected APIs.

2. Workout Management:

Supports full CRUD operations (add, view, update, and delete workouts), with every workout record strictly linked to its authenticated user.

3. Workout Search:

Improves workout history accessibility by enabling users to search and filter records by workout name, category, or date.

4. Google Gemini Integration:

Communicates with Google Gemini AI to generate intelligent workout recommendations, taking user inputs (age, fitness goal, and experience level) to output a weekly workout plan, suggested exercises, training tips, and safety recommendations.

5. AI Fitness Insights:

Analyzes user workout statistics (total workouts, average duration, and calories burned) to produce a performance analysis, improvement suggestions, motivational advice, and a fitness progress summary.

6. Centralized Error Handling:

Returns meaningful, standardized JSON error responses for invalid credentials, missing authentication tokens, validation errors, database errors, and internal server errors.

7. RESTful API Design:

Follows REST principles supporting standard HTTP methods (**GET**, **POST**, **PUT**, and **DELETE**) and returns all responses in JSON format.

8. Modular Architecture:

Enhances maintainability and scalability by separating system responsibilities into Models, Controllers, Routes, Middleware, Services, and Configuration.

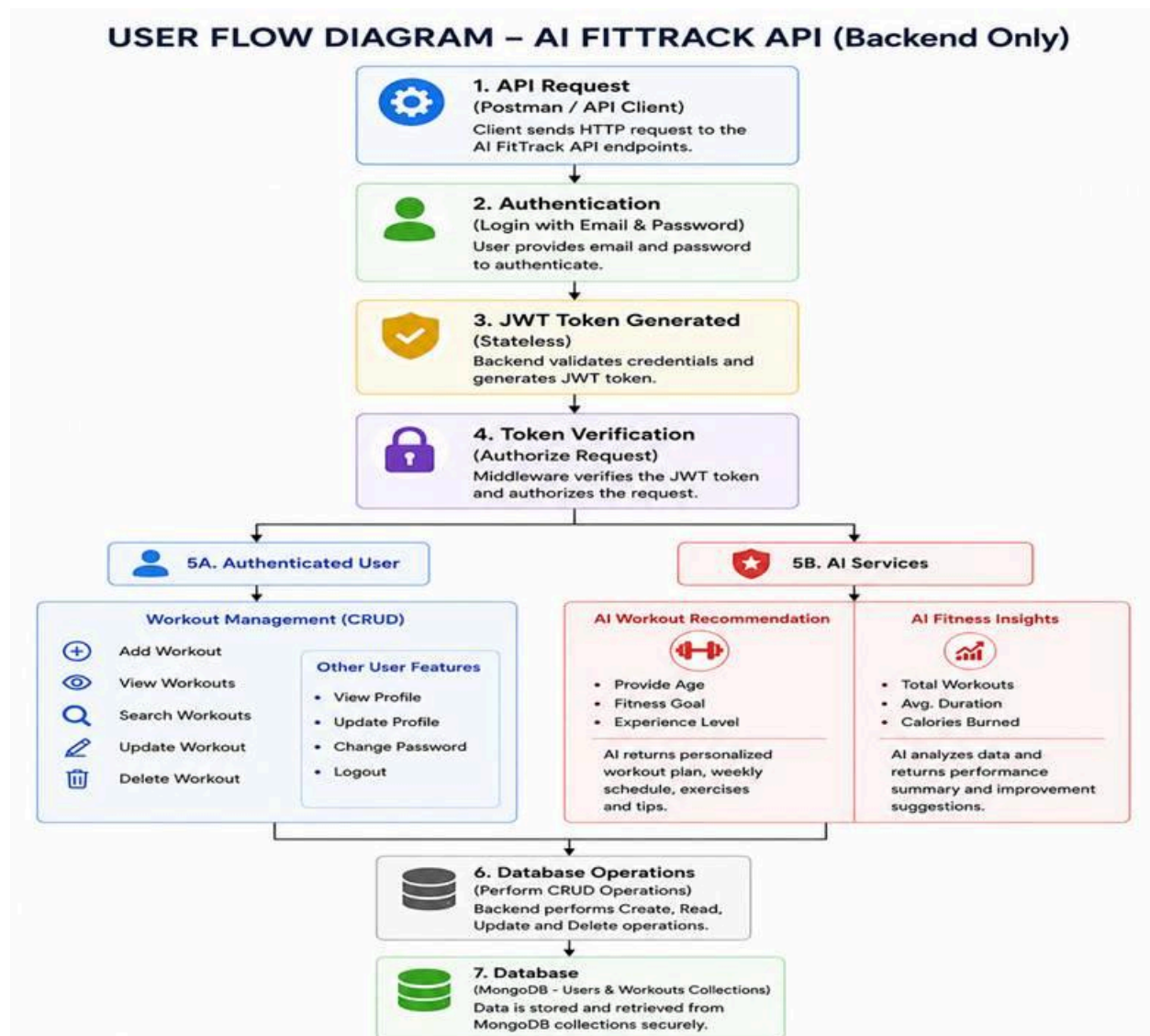
Roles and Responsibilities

The AI FitTrack backend supports one primary role: the **Authenticated User**.

- **Responsibilities & Permissions:**

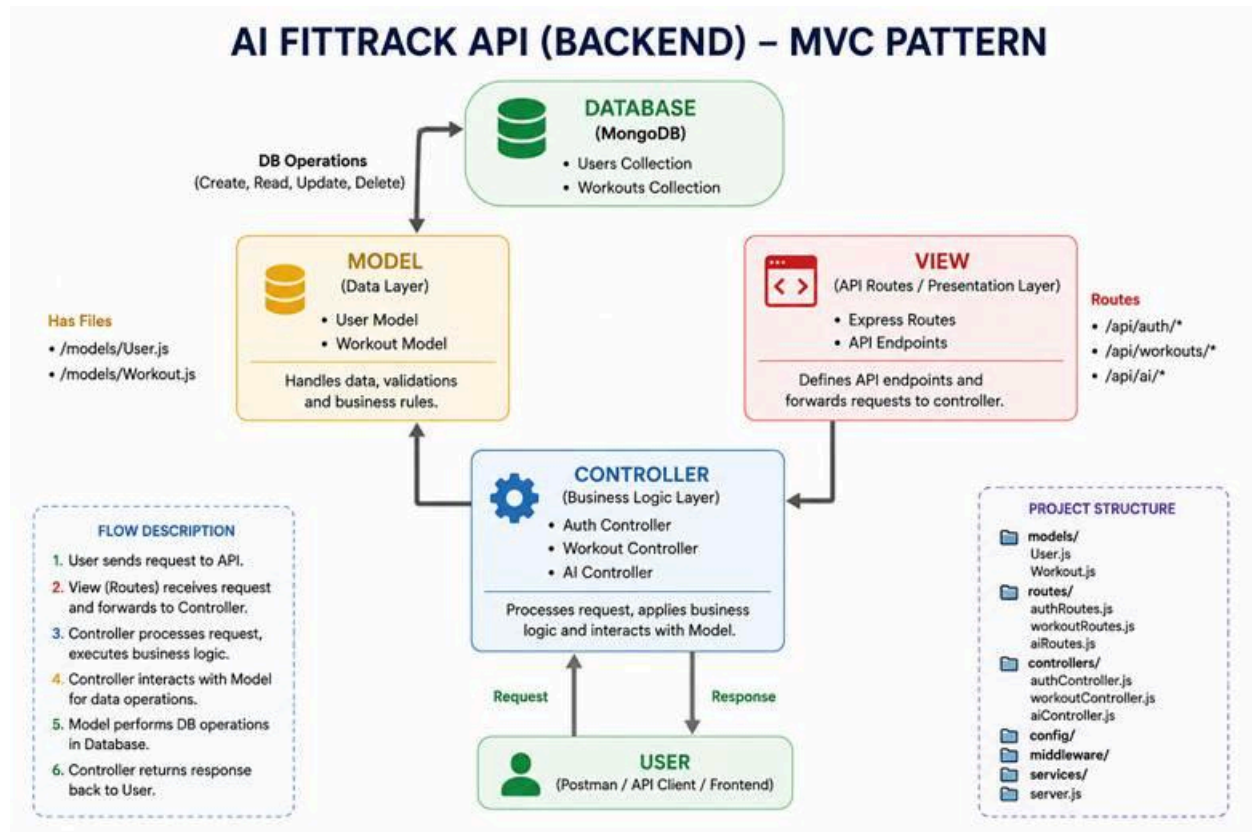
- Authenticated users hold full permissions (**Yes**) across all application operations. This authorizes them to register an account, log in securely, view profile information, perform complete workout management (add, view history, search, update, and delete workouts), and request both AI workout recommendations and AI fitness insights.

User Flow



MVC Architecture Pattern

The FitTrack AI is designed using the **Model-View-Controller (MVC)** architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.



Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

Models in the project include:

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

View Layer (API Interface)

Since AI FitTrack is a backend REST API, there are no traditional UI templates. The View Layer is represented by API consumers such as React applications, mobile applications, or Postman, which interact with the backend through HTTP requests and receive JSON responses.

Responsibilities:

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

Controller Layer (Business Logic)

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

Responsibilities:

- Request Validation
- Business Logic Execution
- Database Interaction
- AI Service Invocation
- Response Generation

The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.

Phase 5

Project Development Phase

Phase 5: Project Development Phase

The FitTrack AI project follows a well-structured backend architecture that separates business logic, routing, database operations, middleware, and AI services into individual modules. This modular design improves code readability, maintainability, and scalability while making the project easier to debug and extend.

The backend is developed using **Node.js** and **Express.js**, with **MongoDB** as the primary database and **Google Gemini AI** for intelligent fitness recommendations.

Creating Project Folder

Step 1: Create the Project Folder

- Create a new folder with the project name: AI-FitTrack-API

Step 2: Open the Project

- Open the project folder in **Visual Studio Code**.

Step 3: Create the Server Folder

- Inside the project directory, create a folder named: **Server**

Step 4: Initialize Node Project

- Open the terminal inside the **server** folder and execute: **npm init -y**

This command generates the **package.json** file containing project metadata and dependency information.

Step 5: Install Required Packages

- Install the required dependencies using:
 - **npm install express mongoose dotenv bcryptjs jsonwebtoken cors**
- Install Nodemon as a development dependency using:
 - **npm install --save-dev nodemon**

Environment Variables

Sensitive information is stored in the **.env** file.

PORT=5000

MONGO_URI=mongodb://localhost:27017/aifittrack

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_google_gemini_api_key

Using environment variables improves security by preventing sensitive information from being exposed in source code.

Backend Development

Backend development focuses on implementing REST APIs that allow authenticated users to manage workouts and interact with Google Gemini AI.

The project is divided into multiple modules that collectively perform authentication, workout management, AI integration, and database operations.

Major modules include:

- Server Configuration
- Database Configuration
- Authentication
- Workout Management
- AI Recommendation
- Middleware
- Error Handling

Database Configuration

The FitTrack AI backend uses MongoDB as its primary NoSQL database to securely store user credentials and workout data, paired with Mongoose as the Object Data Modeling (ODM) library for seamless application-to-database interaction. Together, this database layer maintains data integrity and validates incoming requests while providing schema validation, middleware support, data relationships, and simplified CRUD operations.

MongoDB Configuration

MongoDB stores application data in collections rather than traditional relational database tables. The FitTrack AI maintains separate collections for users and workouts.

Database Collections

- Users Collection
- Workouts Collection

Each collection stores JSON-like documents (BSON format), allowing flexible schema design while maintaining efficient data retrieval.

Installing Mongoose

- Install Mongoose using: `npm install mongoose`

Mongoose acts as the bridge between the Node.js application and MongoDB by providing schema definitions, validation, and database methods.

MongoDB Connection

The database connection is configured inside the **config/db.js** file. This module establishes a connection to the MongoDB server when the application starts.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

- **Responsibilities:** Connects to MongoDB, handles connection errors, displays connection status, and exports the database connection.
- **Advantages:** Provides centralized database configuration, a reusable connection module, automatic connection management, and simplified error handling.

Mongoose Schemas

The AI FitTrack backend contains two primary Mongoose schemas:

1. User Schema
2. Workout Schema

Schemas define the structure, validation rules, and relationships for MongoDB documents.

1. User Schema (models/User.js)

The User schema stores authentication information for registered users.

```
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5    {
6      name: {
7        type: String,
8        required: [true, 'Please add a name'],
9        trim: true,
10     },
11     email: {
12       type: String,
13       required: [true, 'Please add an email'],
14       unique: true,
15       trim: true,
16       lowercase: true,
17       match: [
18         /^\.w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
19         'Please add a valid email address',
20       ],
21     },
22     password: {
23       type: String,
24       required: [true, 'Please add a password'],
25       minlength: 6,
26       validate: [{ validator: function(v) { return v.length >= 6; }, message: 'password must be at least 6 characters' }],
27     },
28     timestamps: true,
29   },
30   {
31     timestamps: true,
32   });
33
34 // Pre-save password hashing hook
35 userSchema.pre('save', async function (next) {
36   if (!this.isModified('password')) {
37     return next();
38   }
39   const salt = await bcrypt.genSalt(10);
40   this.password = await bcrypt.hash(this.password, salt);
41   next();
42 });
43
44 // Compare password method
45 userSchema.methods.matchPassword = async function (enteredPassword) {
46   return await bcrypt.compare(enteredPassword, this.password);
47 };
48
49 module.exports = mongoose.model('User', userSchema);
```

Validation Rules

- Name cannot be empty.
- Email must be unique.
- Password is mandatory.
- Password is encrypted using bcrypt.js before saving.
- Automatic timestamps are enabled.

User Schema Features

- Email uniqueness validation
- Password hashing
- Automatic timestamps
- Secure authentication support
- JWT integration

2. Workout Schema (models/Workout.js)

The Workout schema stores workout information created by authenticated users.

```
src > models > JS Workout.js > workoutSchema > category > enum > values
1  const mongoose = require('mongoose');
2
3  const workoutSchema = new mongoose.Schema(
4  {
5    workoutName: {
6      type: String,
7      required: [true, 'Please add a workout name'],
8      trim: true,
9    },
10   category: {
11     type: String,
12     required: [true, 'Please select a workout category'],
13     enum: {
14       values: [
15         'Cardio',
16         'Strength Training',
17         'Yoga',
18         'Running',
19         'Cycling',
20         'Walking',
21       ],
22       message: '{VALUE} is not a supported workout category',
23     },
24   },
25   duration: {
26     type: Number,
27     required: [true, 'Please add the workout duration in minutes'],
28     min: [1, 'Duration must be at least 1 minute'],
29   },
30   caloriesBurned: {
31     type: Number,
32     required: [true, 'Please specify calories burned'],
33     min: [0, 'Calories burned cannot be negative'],
34   },
35   workoutDate: {
36     type: Date,
37     default: Date.now,
38   },
39   user: {
40     type: mongoose.Schema.Types.ObjectId,
41     ref: 'User',
42     required: true,
43   },
44 },
45 {
46   timestamps: true,
47 }
48 );
49
50 module.exports = mongoose.model('Workout', workoutSchema);
```

Validation Rules

- Workout Name is mandatory.
- Category is mandatory.
- Duration must be greater than zero.
- Calories Burned must be greater than or equal to zero.
- Workout Date is required.
- Every workout belongs to one authenticated user.

Workout Schema Features

- Schema validation
- Automatic timestamps
- User relationship
- Easy search support
- Efficient CRUD operation

Project Execution

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server: **npm start**

Code: [GitHub Repo Link](#)

Phase 6

Project Testing Phase

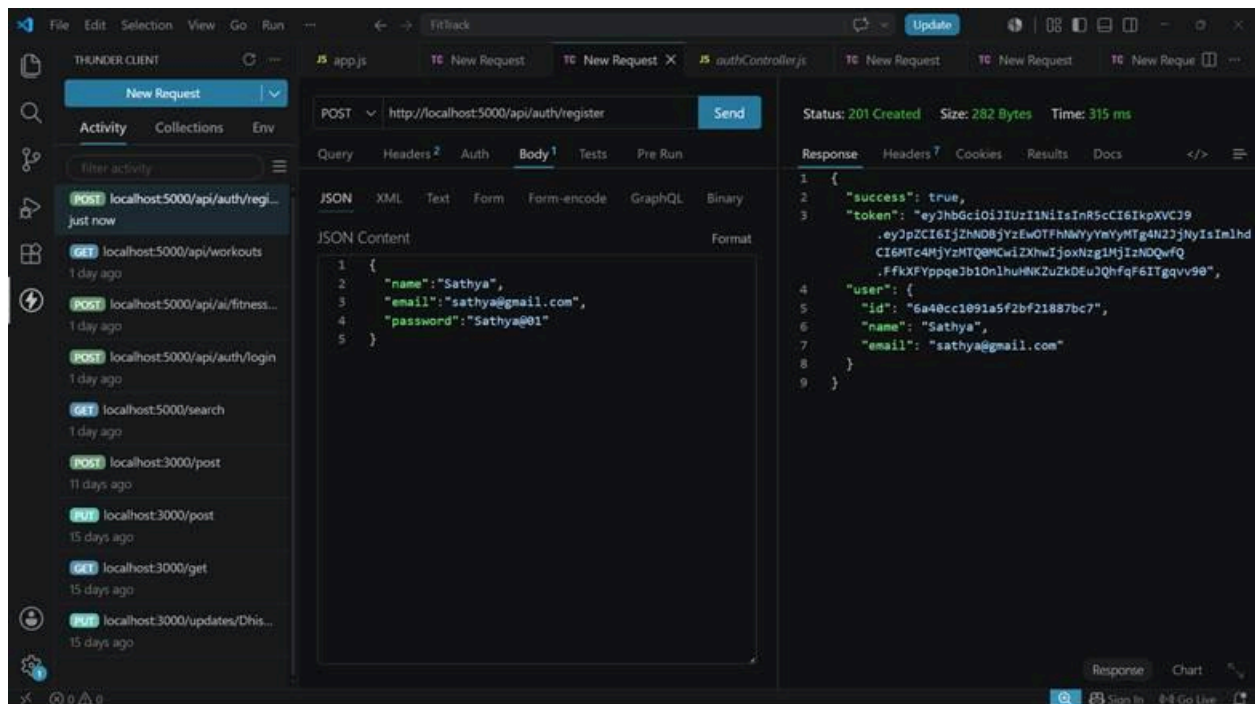
Phase 6: Project Testing Phase

API testing ensures that every endpoint functions correctly and returns the expected response. The FitTrack AI is tested using **Postman**, allowing developers to verify request handling, authentication, database operations, and AI integration.

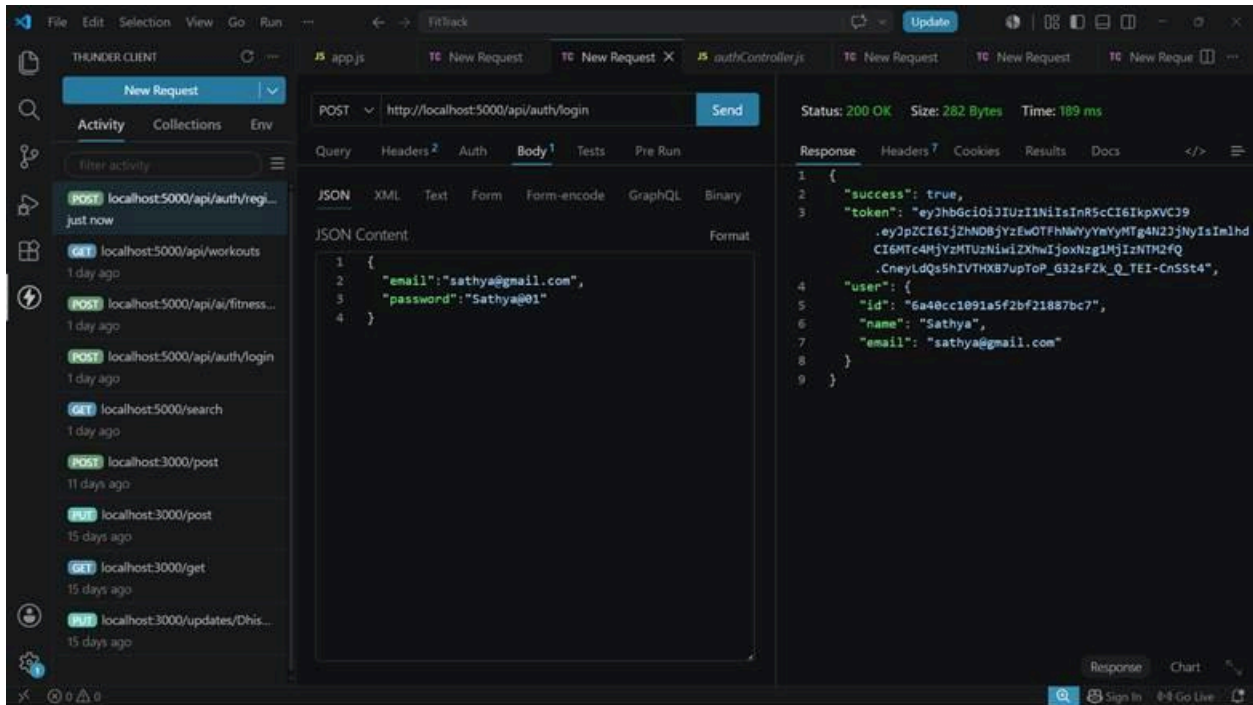
Testing is performed in the following order:

1. Authentication APIs
2. Workout APIs
3. AI APIs

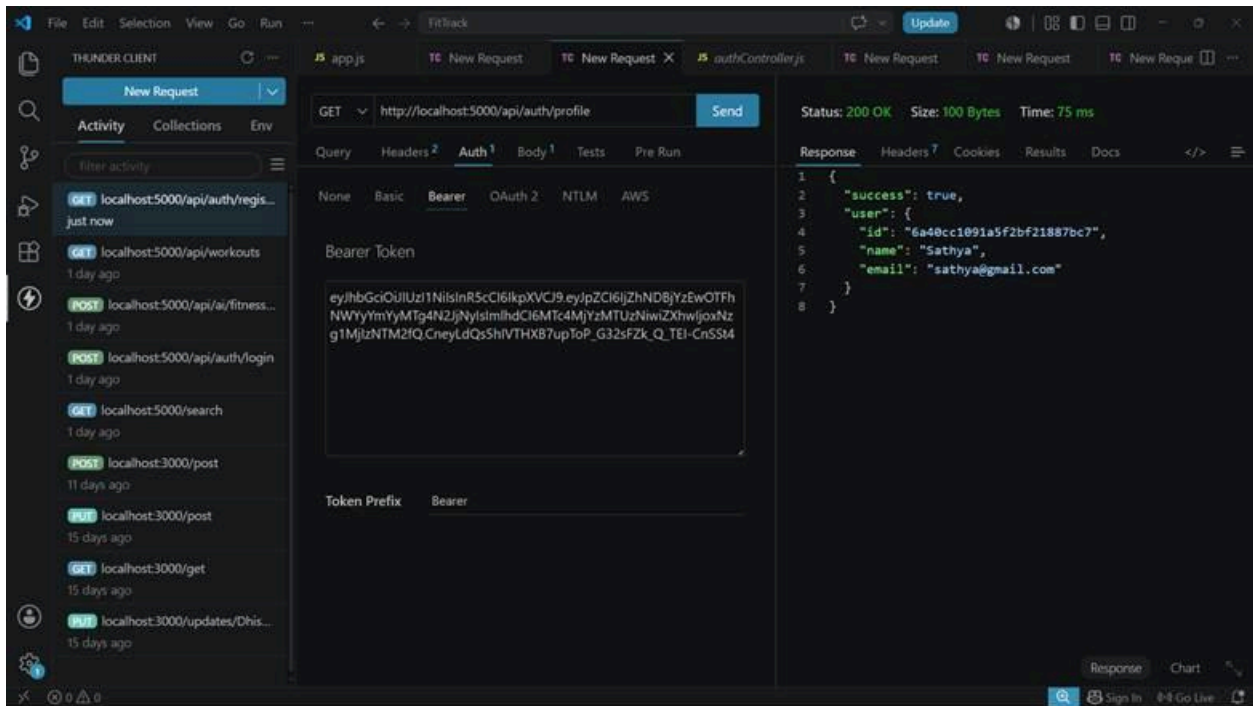
Register User:



Login User:

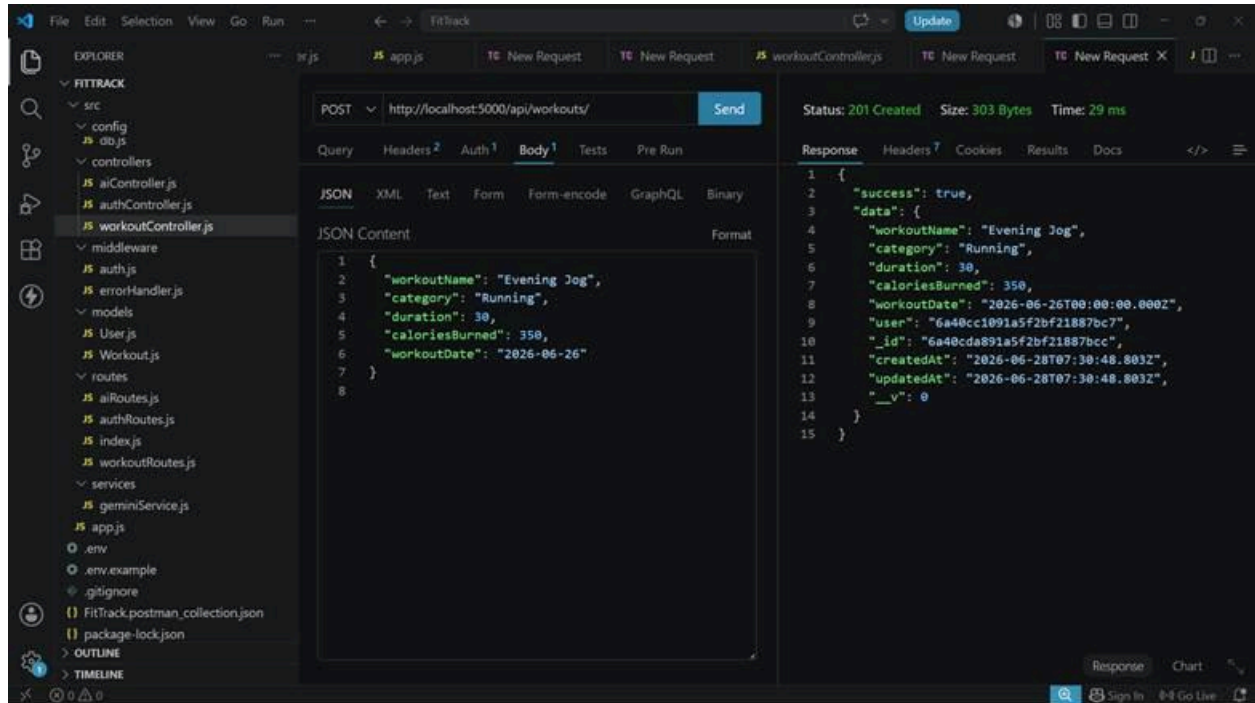


Get Profile:

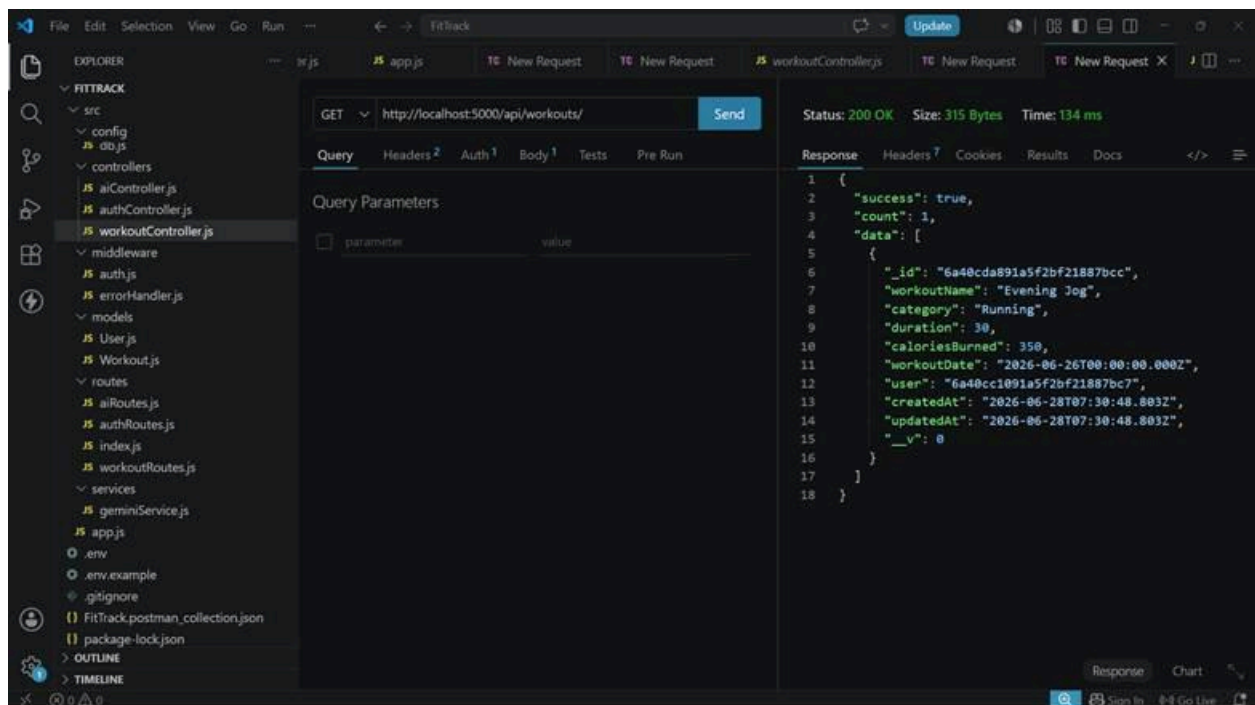


Workout API Testing

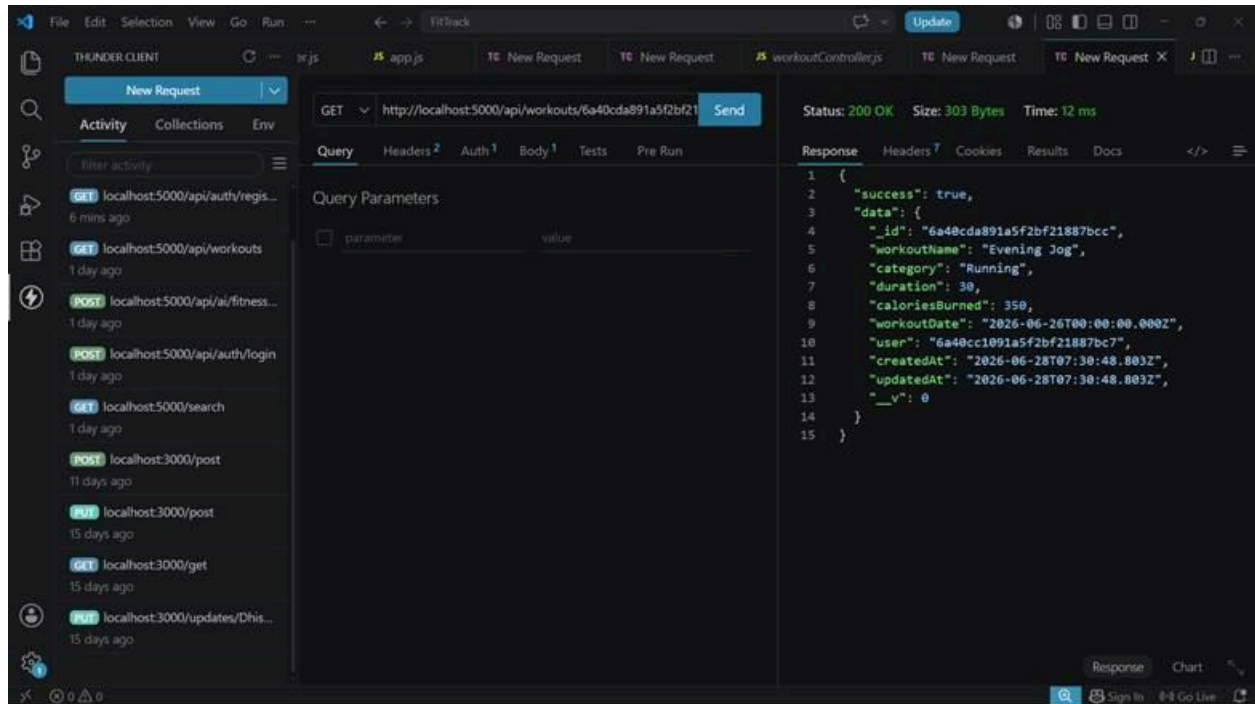
Add Workout:



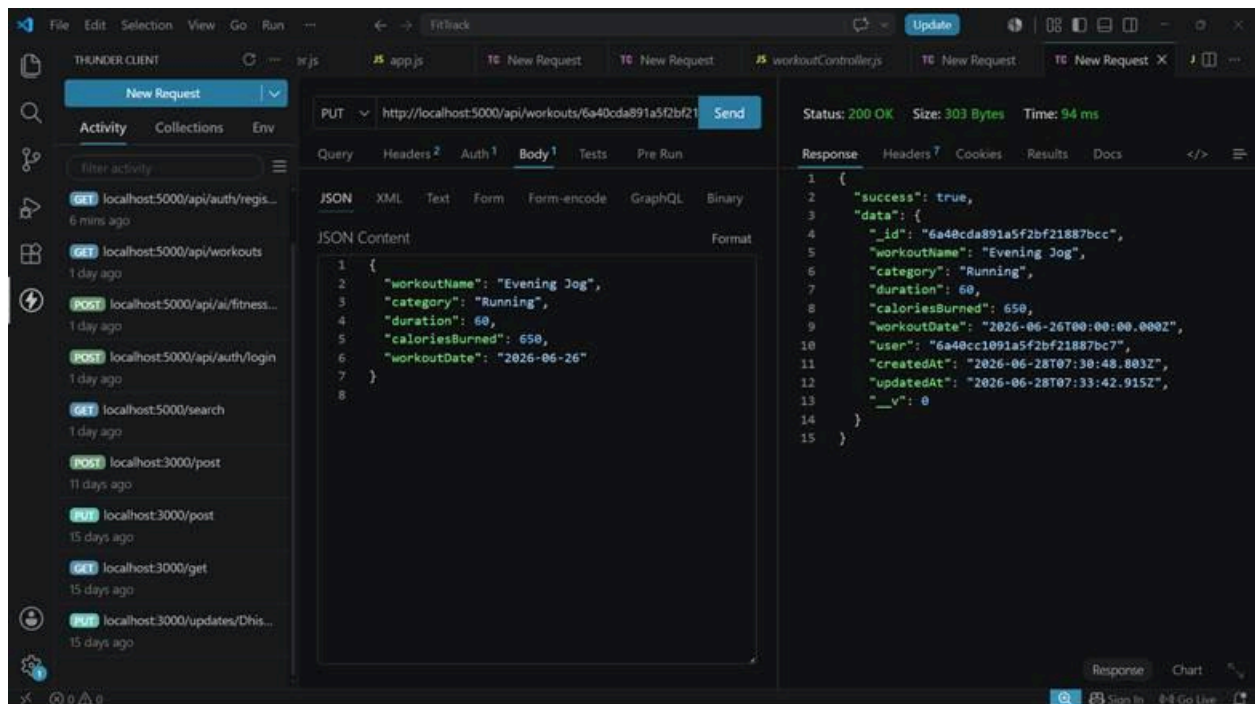
Get All Workouts:



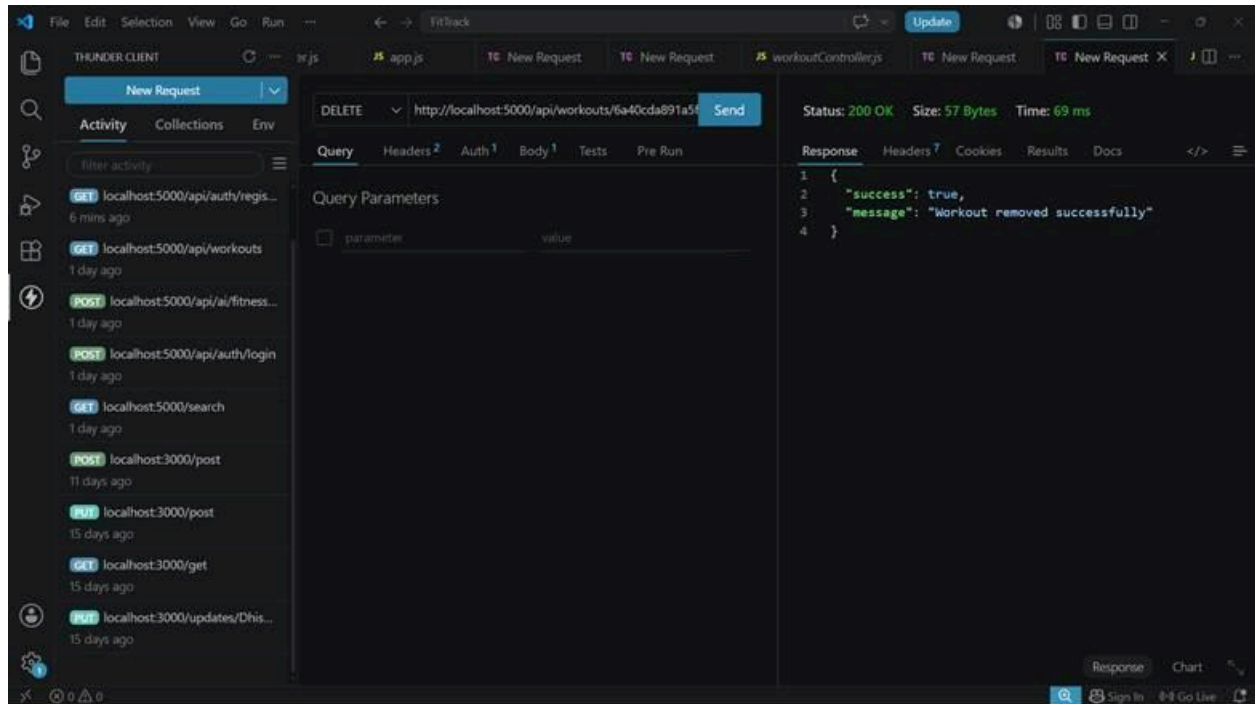
Get Workout By ID:



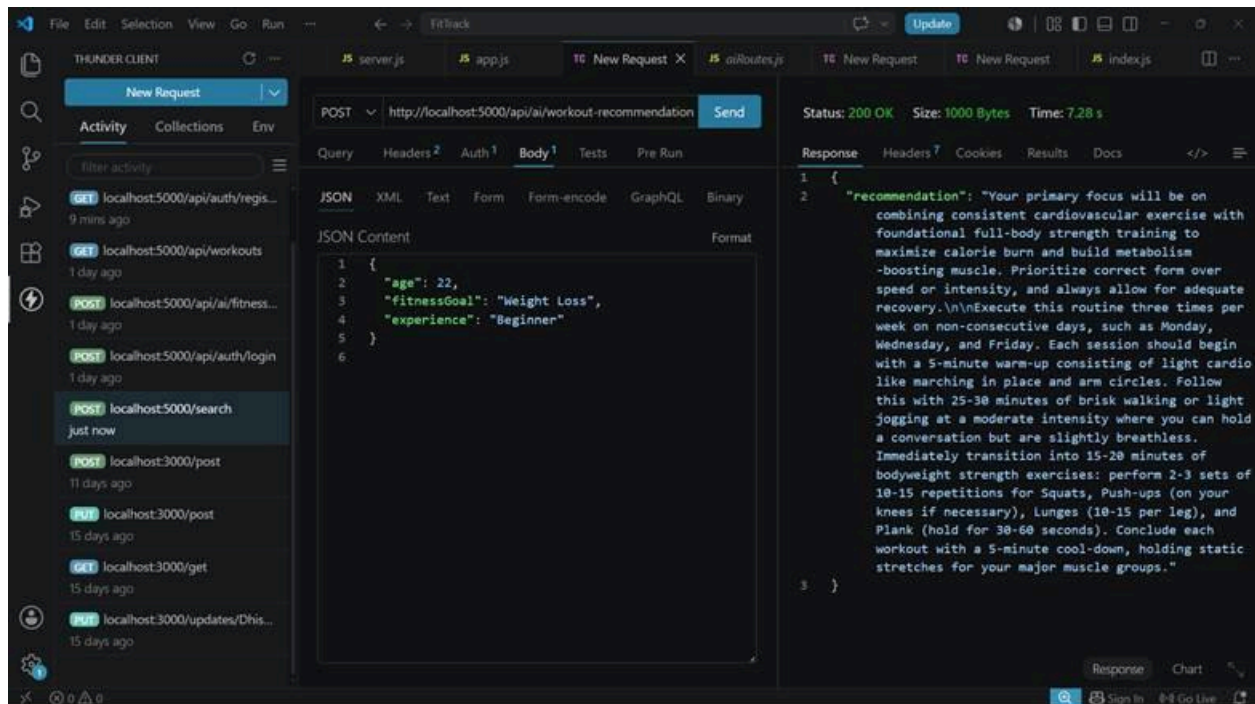
Update Workout:



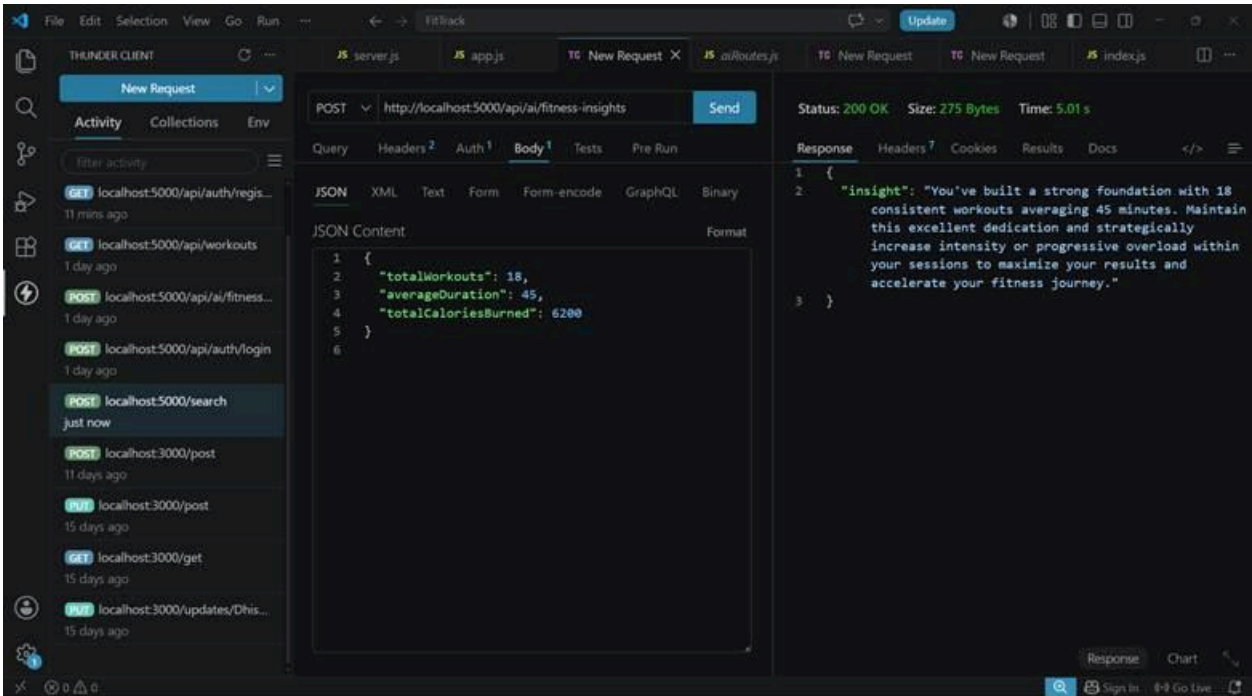
Delete Workout:



Workout Recommendation:



Fitness Insights:



Phase 7

Project Documentation Phase

Phase 7: Project Documentation Phase

Phase	Covers
Brainstorming & Ideation	Manual workout tracking problem & proposed AI fitness solution
Requirement Analysis	Software & hardware requirements
Requirement Analysis	Layered backend architecture & overall system components
Project Planning	Module definitions, ER diagram, user flow & MVC pattern
Project Development	Project setup, MongoDB connection & Mongoose schemas
Project Testing	Authentication, Workout & AI API testing and validation
Project Documentation	Code structure, database schemas & API specifications
Project Demonstration	Server execution & live API request/response verification

- **Problem & Requirements** – Explains the manual workout tracking problem, the proposed FitTrack AI backend solution integrated with Google Gemini AI, and the required hardware and software specifications.
- **Design & Architecture** – Describes the 7-layer backend architecture, ER diagram and database structure, user flow, MVC pattern, authenticated user role, and the one-to-many User-to-Workout relationship.
- **Development & Implementation** – Covers project initialization, environment configuration, Node.js and Express.js backend setup, MongoDB database connection, Mongoose validation schemas (**User** and **Workout**), JWT authentication middleware, and Google Gemini AI integration.
- **Testing & Demonstration** – Covers starting the backend server and testing user registration, login, profile retrieval, workout CRUD operations, and AI-generated workout recommendations and fitness insights using Postman and Thunder Client.

Phase 8

Project Demonstration Phase

Phase 8: Project Demonstration Phase

The project demonstration video provides a complete walkthrough of the FitTrack AI backend application, covering the modular codebase structure, live server execution, and end-to-end REST API testing.

Project File & Architecture Walkthrough:

Explains the Model-View-Controller (MVC) folder structure inside Visual Studio Code. This includes the database configuration (`config/db.js`), Mongoose data models (`models/User.js` and `models/Workout.js`), Express route definitions (`routes/authRoutes.js`, `routes/workoutRoutes.js`, and `routes/aiRoutes.js`), business logic controllers (`controllers/authController.js`, `controllers/workoutController.js`, and `controllers/aiController.js`), JWT authentication and error-handling middleware (`middleware/`), and the Google Gemini service layer (`services/geminiService.js`).

Backend & Database Execution:

Demonstrates configuring the `.env` file with the server port, MongoDB URI, JWT secret, and Google Gemini API key. It then shows starting the Node.js and Express.js server via the terminal (`npm start`) and verifying a successful connection to the local MongoDB database.

Live API Endpoint Testing:

Showcases real-time HTTP request and JSON response verification using Thunder Client and Postman across three core modules:

- **Authentication APIs:** Testing user registration (`POST /api/auth/register`), user login (`POST /api/auth/login`) to generate a stateless JWT token, and protected profile retrieval (`GET /api/auth/profile`) using Bearer token authorization.
- **Workout Management APIs:** Executing protected CRUD operations, including adding a new workout (`POST /api/workouts/`), retrieving all user workouts (`GET /api/workouts`), fetching a specific workout by ID (`GET /api/workouts/:id`), updating workout details (`PUT /api/workouts/:id`), and deleting a workout record (`DELETE /api/workouts/:id`).
- **Google Gemini AI APIs:** Sending user fitness parameters to generate tailored workout plans (`POST /api/ai/workout-recommendation`) and submitting workout metrics to receive AI-analyzed performance summaries (`POST /api/ai/fitness-insights`).

Demo: [Demo Video Link](#)